FLASK CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & APP FACTORY

Installation

```
pip install flask flask-sqlalchemy
def create_app(config=None):
app = Flask(__name__)
app.config.from_object(config)
db.init_app(app)
app.register_blueprint(api_bp, url_prefix='/a...
return app
```

04. TEMPLATES (JINJA2)

Workflow

```
return render_template('index.html', users=us...
{{ user.name | upper }}
{% for u in users %}{{ u.name }}{% endfor %}
{% extends 'base.html' %}
{% block body %}...{% endblock %}
{{ url_for('users', _external=True) }}
```

02. ROUTING

Architecture

```
@app.route('/users', methods=['GET','POST'])
def users():
if request.method == 'POST':
data = request.get_json()
return jsonify(data), 201
return jsonify(User.query.all())
@app.route('/users/<int:user_id>')
```

05. BLUEPRINTS

CRUD API

```
api_bp = Blueprint('api', __name__)
@api_bp.route('/users')
def get_users(): ...
# In factory:
app.register_blueprint(api_bp, url_prefix='/a...
```

Separate modules by feature; maintain isolation

03. REQUEST & RESPONSE

YAML / Config

```
request.method  request.args  request.form
request.json  request.get_json()
request.files  request.headers
jsonify({'key': 'value'})
make_response(body, 200, {'X-Custom': 'val'})
redirect(url_for('index'))
abort(404) / abort(401)
```

06. FLASK-SQLALCHEMY

Integration

```
db = SQLAlchemy()
class User(db.Model):
id = db.Column(db.Integer, primary_key=True)
email = db.Column(db.String(120), unique=True)
User.query.filter_by(active=True).all()
db.session.add(user) / db.session.commit()
flask db upgrade  (Flask-Migrate)
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ERROR HANDLING

Hardening

```
@app.errorhandler(404)
def not_found(e):
return jsonify(error='Not found'), 404
@app.errorhandler(Exception)
def handle_exception(e):
return jsonify(error=str(e)), 500
try_except + abort() for controlled errors
```

08. SESSIONS & COOKIES

Observability

```
from flask import session
session['user_id'] = user.id
session.get('user_id')
session.clear()
app.secret_key = 'change-me'
response.set_cookie('token', value, httponly=...
```

Session signed server-side with secret_key

09. BEFORE/AFTER REQUESTS

Optimization

```
@app.before_request
def auth_check():
if not session.get('user_id') and request.end...
return jsonify(error='Unauthorized'), 401
@app.after_request
def add_headers(response):
response.headers['X-Frame-Options'] = 'DENY'
return response
```

10. CONFIGURATION

Unit Tests

```
app.config.from_pyfile('config.py')
app.config.from_envvar('APP_SETTINGS')
app.config['DEBUG'] = os.environ.get('DEBUG',...
class ProductionConfig:
SQLALCHEMY_DATABASE_URI = os.environ['DATABAS...
SECRET_KEY = os.environ['SECRET_KEY']
```

11. POPULAR EXTENSIONS

Commands

```
Flask-SQLAlchemy  ORM integration
Flask-Migrate     Alembic migrations
Flask-Login       user session management
Flask-JWT-Extended  JWT tokens
Flask-WTF         CSRF + form validation
Flask-CORS        CORS headers
Marshmallow       serialization/validation
```

12. COMMON GOTCHAS

Warning

Application context: push ctx for DB outside request
Thread safety: don't store state on app object
SECRET_KEY must be set or sessions won't work
FLASK_ENV=development enables debugger
Use factory pattern for testing avoid module-level app